

Sensitivity Experiment Tips for Teammates

- This note explains how to run the version 2 sensitivity experiment code on a different computer.
- Main script: `run_sensitivity.py`
- Experiment settings: `configs/*.yaml`
- Recommended first step: run a tiny test before launching a large experiment.

1. Change File Paths First

- Each YAML config contains machine-specific paths. Before running, edit the experiment section.
- Set data_root to the extracted Speech Commands dataset folder, for example:
"/Users/yourname/data/speech_commands".
- Set tarball to the downloaded speech_commands_v0.02.tar.gz file if the dataset is not extracted yet.
- Set output_dir to where you want results saved. Relative paths such as "outputs/representation" are saved inside the version 2 folder.
- The data_root folder should contain class folders such as yes, no, up, down, left, right, stop, and go.
- If a path contains spaces, keep it inside quotes in the YAML file.

2. Use Your Own Python Environment

- The original machine used `/opt/anaconda3/envs/speech311/bin/python`, but your path may be different.
- Use a Python or conda environment that has these packages: `librosa`, `soundfile`, `openai-whisper`, `pyyaml`, `pandas`, `numpy`, and `matplotlib`.
- Quick check: `python -c "import librosa, soundfile, whisper, yaml, pandas, matplotlib; print('OK')"`
- If using conda: `conda activate your_env_name`
- Then run: `python run_sensitivity.py --config configs/representation.yaml`
- Whisper may download a model the first time it runs, so internet/model-cache access may be needed.

3. Choose Folders and Number of Audio Files

- In the YAML config, edit labels, files_per_label, selection, max_files, and seed.
- Example: labels: ["yes", "no"]
- files_per_label: 4 means four WAV files from each selected folder.
- selection: first uses the first files alphabetically.
- selection: random samples randomly but reproducibly using seed.
- max_files is an optional total cap across all folders. Leave it blank if you do not need a total cap.
- Example larger run: labels: ["yes", "no", "up", "down", "stop"], files_per_label: 100, selection: random.

4. Change Which Parameter Is Tested

- The parameters section controls one-parameter-at-a-time sweeps.
- Example: name: frame_size_ms, values: [10, 15, 20, 25, 30, 40, 50, 75, 100].
- To test only n_mels, keep one block with name: n_mels and values: [16, 24, 32, 40, 48, 64, 80, 100, 128].
- You can include multiple parameter blocks, but the script still changes one parameter at a time.
- It does not test all parameter combinations. Combination testing is a later stage.

5. Config Files

- configs/representation.yaml: frame size, hop size, mel bins, frequency cutoffs, MFCC, and frequency masks.
- configs/compression_method.yaml: log mel, normalization, dynamic range clipping, quantization, and time downsampling.
- configs/robustness.yaml: SNR and noise type.
- configs/pipeline.yaml: reconstruction method, Griffin-Lim iterations, and Whisper model size.

6. Result Files

- After a run, check the output_dir folder.
- raw_results.csv: one row per audio file per parameter value.
- summary_by_value.csv: average accuracy, WER, compression rate, waveform SNR, and mel MSE for each value.
- change_efficiency.csv: accuracy drop, compression gain, and change efficiency compared with baseline.
- recommendations.md: text summary suggesting useful ranges and increment advice.
- plots/: graphs of accuracy and compression rate against the tested parameter.

7. Useful Reminders

- `frame_size_ms` changes reconstruction and ASR quality, but usually not compression rate, because the number of frames is mainly controlled by `hop_ms`.
- `hop_ms`, `n_mels`, `quant_bits`, `time_downsample_factor`, and `n_mfcc` are the most direct compression-rate parameters.
- For large runs, set `save_recon_audio: false` unless you really need every reconstructed WAV.
- Use `whisper_model`: `tiny` for testing speed, then `base` or larger models for more serious evaluation.
- Start small, confirm the output files look correct, then scale up.